socrative
A showbie PRODUCT

Answer Key
October 26, 2026

ONEM Spring day 2

1. Why is constructor injection preferred over field injection?

A It is faster at runtime

✓ B Dependencies become required, the field can be final, and a missing dependency fails early instead of causing a late NullPointerException

C Spring only supports field injection for prototype beans

D It removes the need for @Component

i Required parameters, a final field, and a failure at compile time instead of a late NPE.

2. A prototype-scoped bean is injected into a singleton. What happens?

A The singleton gets a new prototype instance on every method call

B Spring throws an exception at startup

✓ C The prototype is resolved once, so the singleton keeps the same instance. Use an ObjectProvider for a fresh one each time

D The prototype automatically becomes a singleton for the whole application

i Injection happens only once, when the singleton is created. ObjectProvider gives a fresh instance on every call.

3. What does a "no unique bean" error mean?

✓ A More than one bean matched the required type. Fix it with @Primary or @Qualifier

B No bean matched the required type. Fix it by adding @Component

C Two beans depend on each other (circular dependency)

D The active profile does not match

i Too many beans matched. @Primary sets a default, @Qualifier is explicit every time.

4. When does a @PostConstruct method run?

A Before the constructor

B Every time a method of the bean is called

C At shutdown, before the bean is discarded

✓ D Once, right after the bean is fully wired

i It is the setup hook that runs once after wiring. @PreDestroy is the shutdown hook.

5. What happens when a @Transactional method is called from another method in the same bean (self-invocation)?

A Spring throws a BeanCreationException at startup

✓ B The call bypasses the proxy, so @Transactional is silently ignored

C Two transactions are started

D The method rolls back automatically

i The proxy is bypassed with no error at all. Fix it by extracting the method into a separate bean.

6. You load a managed entity in a @Transactional method and change a field, but never call save(). Does the change reach the database?

A No, you must always call save()

B Only if you call flush() manually

✓ C Yes, dirty checking detects the change at flush/commit and issues the UPDATE

D Only if the method is annotated with @Modifying

i The persistence context compares the entity to its loaded snapshot at flush/commit.

7. What happens when you change a detached entity?

✓ A Nothing: the change is silently lost, with no exception

B It is automatically reattached and saved in the next transaction

C An exception is thrown immediately

D The change is written when the application shuts down

i A detached entity is just a plain object, so nobody tracks the change.

8. What is the difference between flush and commit?

A They are the same thing

✓ B Flush sends the pending SQL to the database inside the transaction, commit makes it permanent

C Flush makes changes permanent, commit sends the SQL

D Flush clears the persistence context, commit never touches the database

i Flush sends the SQL, commit makes it permanent.

9. By default, how does Spring handle a checked exception thrown from a @Transactional method?

- A It rolls back, nothing extra is needed
- B It rolls back only when the isolation level is SERIALIZABLE
- C It marks the transaction rollback-only but still commits the other changes
- ✓ D It commits. Add rollbackFor to roll back on a checked exception

i Only unchecked exceptions (RuntimeException and Error) trigger a rollback by default.

10. A REQUIRED method calls another REQUIRED @Transactional method that throws. The caller catches the exception. What happens at the end?

- A The outer transaction commits normally because the exception was caught
- B The inner work rolls back alone and the outer work commits
- ✓ C The commit fails with an UnexpectedRollbackException, because the shared transaction was already marked rollback-only
- D Spring retries the inner method in a new transaction

i Both methods share one transaction, and the inner failure already marked it rollback-only.

11. What does REQUIRES_NEW do?

- ✓ A It suspends the caller's transaction and starts an independent one. The outer transaction still holds its connection meanwhile
- B It joins the caller's transaction
- C It runs the method without any transaction
- D It creates a savepoint that rolls back together with the caller

i The inner transaction commits independently, while the outer one is only suspended and keeps its resources.

12. Two users load the same customer, each changes a different field, and both save. The first change is silently overwritten. What is this, and what is the standard fix?

- A A dirty read, fixed with REQUIRES_NEW
- ✓ B A lost update, fixed with optimistic locking (@Version)
- C A deadlock, fixed with shorter timeouts
- D The N+1 problem, fixed with a fetch join

i Last writer wins. With @Version the second save fails loudly instead of overwriting.

13. Why does wrapping “check stock, then reserve it” in @Transactional not stop two concurrent orders from both taking the last item?

A @Transactional cannot be combined with JPA

B The transaction is too short and should be extended

C You need REQUIRES_NEW on both methods

✓ D At the default isolation level another transaction can read the same stale value between check and update. Use one atomic UPDATE … WHERE quantity >= :amount

i One atomic statement plus a check of the rows affected is the robust fix.

14. Why can a database transaction and sending a message to a broker not be made atomic together?

✓ A They are two separate systems, each with its own commit and no coordinator. So you make failure harmless (publish-after-commit, outbox)

B Spring forbids sending messages inside @Transactional methods

C Brokers always deliver exactly once, so the problem does not exist

D Hibernate flushes break messaging

i There is no shared transaction, so you design for failure instead of trying to make both atomic.

15. What is an idempotent consumer, and why do you need one even with an outbox?

A A consumer the broker guarantees will receive each message exactly once

✓ B A consumer that can process the same message twice without extra effect, needed because delivery is at-least-once and duplicates will happen

C A consumer that never throws exceptions

D A consumer that reads messages from the outbox in strict order

i At-least-once delivery means duplicates, so the consumer must absorb them safely.